

Prog 4 – pótpótZH

A feladatsort figyelmesen olvasd el!

A ZH-t zip fájlként kell feltölteni az alábbi formátumban:

név_neptun_kód_prog4_potpotzh.zip

Az alkalmazáserver helye a labor gépein: [C:\apache-tomcat](#)

A ZH feladat leírása és docker-compose fájlja mellett található egy **auditalo.exe** állomány, amit első lépésként tölts le.

Hozz létre egy mappát az alábbi néven: **<név_neptun_kód>_prog4_potpotzh**

Ebbe a létrehozott mappába másold be az **auditalo.exe** állományt, majd indítsd el, ez létre fog hozni két mappát: **zh**, **zh_audit**

Ha a Windows blokkolná a futtatást, akkor kattints a felugró ablakban a **További információ** linkre, majd a **Futtatás mindenképpen** opcióra, ezután megnyílik egy command window, **ezt ne zárd be, a teljes ZH alatt futnia kell.**

Ezt követően a **zh** mappába hozd létre a feladatoknak megfelelő alkalmazásokat és itt dolgozz (innen nyisd meg Intellij Idea-ban).

A **zh_audit** mappa tartalmához TILOS nyúlni és annak módosítása is TILOS.

Nem forduló kód (maven build használatával) vagy el nem induló alkalmazás automatikusan 0 pontos ZH-t eredményez.

ChatGPT (vagy bármilyen AI) és chat alkalmazások megnyitása vagy használata, illetve egymás segítése tilos, ennek észrevétele (akár a ZH során, akár a beadott megoldás alapján) szintén 0 pontos ZH-t eredményez.

Az auditáló futtatásának hiánya, annak idő előtti leállítása, a fájljainak módosítása 0 pontos ZH-t eredményez.

Órai anyag és internet használható a feladatok megoldása során, viszont teljes blokkok másolása esetén az adott feladat 0 pontot ér (ez pom.xml-re is vonatkozik, csak az legyen behúzva, amire az adott alkalmazásnak szüksége van).

A 0. feladat git kezeléshez fog kapcsolódni, az ott leírtak beugrónak számítanak.

Git repo hiánya esetén a ZH 0 pontos, commitálás nélkül beadott kód 0 pontot ér.

A teljes ZH egyben történő commitálása esetén a ZH szintén 0 pontos.

Több feladat egyben történő commitálása esetén csak az egyik feladat lesz figyelembe véve a javítás során.

Mivel a ZH jelentős része kapcsolódik az adatbázishoz és az adatok is ott kerülnek tárolásra, így az adatbázis kapcsolat hiánya esetén a ZH 0 pontot ér.

A **zh** mappán kívüli található megoldások nem lesznek figyelembe véve, arra 0 pont jár.

Beadáskor egyetlen tömörített (ZIP) állományt kell feltölteni. Ehhez a korábban létrehozott <név_neptun_kód>_prog4_potpotzh mappát tömörítsd. A feltöltött állománynak tartalmaznia kell a zh és zh_udit mappákat, valamint az auditáló programot.

Ellenőrizd, hogy a mariadb-java-client-3.1.3.jar megtalálható-e a tomcat alatt, ha nem akkor a gyakorlaton mutatott módon töltsd le az internetről és helyezd el a megfelelő könyvtárban.

A feladathoz szükség van egy adatbázisra. Amennyiben elindítod a projektben található compose-t, akkor elindul az adatbázis és minden szükséges objektum létrejön benne.

Az adatbázis kapcsolatokhoz szükséges resource-ok definíciója, **amit szükséges kiegészíteni a hiányzó értékekkel** (ha már van másik resource felvéve azt töröld ki):

Adat resource:

```
<Resource name="jdbc/PotPotZH_university_MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/potpotzh_university_database"
username="potpotzh_university_admin_user" password="PotPotZH2026Password111"
maxActive="20" maxIdle="10" maxWait="-1" />
```

Security resource:

```
<Resource name="jdbc/PotPotZH_security_MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/potpotzh_security_database"
username="potpotzh_security_admin_user" password="PotPotZH2026SecurityPassword111"
maxActive="20" maxIdle="10" maxWait="-1" />
```

Az adatbázisban találhatóak felhasználók és szerepkörök is.

A felhasználó jelszava "Password111". A jelszó Bcrypttel van elforgatva.

A beadandónak két alkalmazást kell tartalmaznia:

- potpotzh-feladat
- potpotzh-jsp-feladat

Rövid ismertető az alkalmazásokról:

Az alkalmazások hallgatókra vonatkozó adatokat tárolnak.

Két külön alkalmazásunk van.

Az egyik alkalmazásunk kommunikál az adatbázissal (lehet lekérdezni listát, azonosító alapján, létrehozni új hallgatót és törölni), található benne egy service réteg minden repository metódushoz.

Ennek az alkalmazásnak van egy controller rétege, ami api endpointokat tartalmaz a service rétegben található metódusokhoz.

Ennek az alkalmazásnak van egy felülete, ami listázza az adatbázis táblában található adatokat, a felületet a servlet rendeli ki (nem jsp!) a hallgatók neve szerinti abc sorrendben.

Az alkalmazásban található egy webservice, amin keresztül az adatbázis tábla tartalmát lehet lekérdezni, új hallgatót létrehozni, illetve hallgatót törölni.

Ennek az alkalmazásnak nincs security-je.

A másik alkalmazás az adatokat az előző alkalmazásban kiadott webservicen keresztül éri el, illetve módosításokat is azon keresztül tud végezni az adatokon.

Ebben az alkalmazásban található egy listázó felülete, ahol a hallgatókat lehet megjeleníteni, itt a lista mellett található egy törlés gomb és egy xml gomb, ami console-ra logol.

A listázó felület alatt található egy legjobb hallgató gomb, ami kigyűjti a legjobb átlaggal rendelkező hallgatót és megjeleníti a felületen.

Van egy új hallgató rögzítése felület is, ahol új hallgatót lehet rögzíteni.

Ez az alkalmazás security-t használ, a létrehozó oldalt csak a létrehozó szerepkörrel rendelkező felhasználó érheti el, a törlést pedig a törlő szerepkörrel rendelkező, a listázó oldalhoz és az xml generáláshoz pedig mindegyiküknek van jogosultsága.

Feladat leírás:

Git (potpotzh-feladat és potzh-jsp-feladat)

- Hozz létre egy git repository-t, majd ebbe a repository-ba hozd létre az alkalmazásokat is.
- Készíts egy develop branchet, majd ezen a branchen dolgozz.
- A kapott projekt fájlokat commitáld el egyben.
- Minden feladat megoldását külön-külön commitáld el, figyelj a beszédes commit message-kre.

Hozd létre a feladat leírásában szereplő alkalmazásokat, mind a kettő tomcat szerveren web alkalmazásként futtatható legyen.

Az első alkalmazásban az adatbázis táblákat használva hoz létre a repository réteget (lehet lekérdezni listát, azonosító alapján, létrehozni új hallgatót, illetve törölni).

Készítsd el hozzájuk a service rétegbeli metódusokat.

Készíts egy webservice-t, ami visszaadja a hallgatók listáját, azonosító alapján kereséshez, törléshez és egy létrehozáshoz használható metódust.

Készítsd el az api endpointokat a service rétegben található metódusokhoz.

Készítsd el a listázó felületet, ahol megjelennek a hallgatók név szerinti ABC sorrendben, a felület rendelését a servlet végezze (ne jsp legyen!).

Készítsd el a második alkalmazást.

A korábban elkészített webservicet használva valósítsd meg az alkalmazás funkcionalitását (minden hallgatóhoz köthető adatbázis művelet webservice-n keresztül történjen).

Legyen egy felület, ahol listázni lehet a hallgatókat, mellette legyen egy törlés és egy xml gomb.

Törlés esetén az adott hallgató kerüljön törlésre az adatbázisból.

Az xml gombra kattintva logolja ki console-ra xml formátumban az adott hallgató adatait.

Legyen a listázó gomb alatt egy legjobb átlag gomb, ami a legjobb átlaggal rendelkező hallgató nevét és szakját írja ki a felületre tag library-t használva (tag-ből ne hívj servicet!).

Legyen egy új hallgató rögzítő felület, ahol új hallgatót lehet rögzíteni.

Ebben az alkalmazásban a listázó és a rögzítő oldal alján is legyen egy page, ahol navigálni lehet az oldalak között, illetve legyen egy kijelentkezés link is, ami a kijelentkezést végzi el.

Készítsd el az autentikációt erre az alkalmazásra:

FORM alapú autentikáció legyen.

A webservice legyen elérhető bejelentkezés nélkül.

Törölni csak a törlő szerepkörrel rendelkező user tudjon.

Létrehozni csak a létrehozó szerepkörrel rendelkező user tudjon.

A listázó oldal, az xml generálás, illetve a legjobb hallgató funkció legyen elérhető mindegyik szerepkör számára.

Legyen hibás bejelentkezésnek saját oldala.